

MANUAL TÉCNICO · BLUE TEAM & RED TEAM

SENTINEL LAB

Construção de um Security Operations Center doméstico: arquitetura, instalação passo a passo, erros encontrados e lições aprendidas.

Proxmox

Wazuh

Suricata

OpenVPN

AdGuard Home

Honeypots

OpenWRT

Craveiro · github.com/craveirorj

Endereços de rede anonimizados para publicação — ver Capítulo 1

Aviso sobre os dados apresentados neste manual

Este documento descreve um laboratório de cibersegurança pessoal, real e em funcionamento. Como se destina a publicação aberta (GitHub), **todos os endereços IP, nomes de domínio e identificadores de rede foram substituídos por valores fictícios**, escolhidos apenas para ilustrar a arquitetura de forma coerente e fácil de seguir.

Sempre que um endereço IP de exemplo aparece no texto ou num bloco de código, está destacado a cores desta forma: **192.168.50.100** — isto assinala visualmente que se trata de um valor ilustrativo, e não do endereço real da rede do autor.

Nenhuma password, chave privada, token ou segredo real está, ou esteve alguma vez, presente neste documento.

Índice

Capítulo 1	Introdução e Visão Geral	4
Capítulo 2	Equipamento e Hardware	7
Capítulo 3	Proxmox VE: a Base de Tudo	14
Capítulo 4	Rede: Switch, Router e Segmentação WiFi	19
Capítulo 5	AdGuard Home: Filtragem e Visibilidade de DNS	25
Capítulo 6	Wazuh (SIEM/XDR) e Suricata (IDS)	28
Capítulo 7	Honeypots: Cowrie e OpenCanary	32
Capítulo 8	A Rede Isolada de Alvos Vulneráveis	36
Capítulo 9	Acesso Remoto Seguro: OpenVPN	39
Capítulo 10	Alertas de Dispositivos e Resposta Automática	48
Capítulo 11	SENTINEL AI: um Analista Assistente	51
Capítulo 12	Hack The Box: Prática Externa e Progressão Reconhecida	53
Capítulo 13	Lista Consolidada de Erros e Correções	55
Capítulo 14	Próximos Passos: a Fase de Testes	59

Capítulo 1 — Introdução e Visão Geral

O que é um SOC, e porque construir um em casa

Um **SOC (Security Operations Center)** é, em qualquer organização, a estrutura — pessoas, processos e ferramentas — responsável por vigiar continuamente redes e sistemas, detetar atividade suspeita, investigar alertas e responder a incidentes antes (ou depois) de causarem dano real. Numa empresa grande, isto significa dezenas de analistas a olhar para painéis de um SIEM (Security Information and Event Management), sondas de deteção de intrusão espalhadas pela rede, e processos formais de resposta a incidentes.

Este projeto — **SENTINEL LAB** — é a tentativa de recriar, à escala de uma rede doméstica, exatamente esse conjunto de capacidades: um SIEM real (Wazuh), um sistema de deteção de intrusão real (Suricata), isco para atacantes (honeypots), máquinas deliberadamente vulneráveis para praticar ataques (Metasploitable2, Kioptrix, DVWA), uma rede segmentada com propósito, VPN para acesso remoto seguro, e um assistente de IA próprio para ajudar na análise de alertas. Tudo isto corre em hardware de consumidor, dentro de casa, sobre uma plataforma de virtualização (Proxmox VE) que permite ligar e desligar cada peça como se fossem servidores independentes.

A motivação é simultaneamente de aprendizagem e de prática deliberada: ferramentas de cibersegurança só se dominam a sério quando se instalam, configuram, partem, e se corrigem os próprios erros — não apenas quando se lê sobre elas. Este manual documenta exatamente esse processo, incluindo os erros, becos sem saída e decisões de desenho que normalmente não aparecem em tutoriais "perfeitos".

Blue Team vs. Red Team — os dois lados que este lab cobre

Blue Team é a designação para quem defende: monitoriza, deteta, responde. **Red Team** é quem ataca — de forma autorizada e controlada — para testar se as defesas realmente funcionam. Um profissional de cibersegurança completo compreende ambos os lados, porque não se sabe defender bem aquilo que nunca se tentou atacar, e vice-versa. O SENTINEL LAB foi desenhado com os dois papéis em mente: as máquinas vulneráveis e as contas em plataformas de treino (Hack The Box) representam o lado Red Team; o Wazuh, o Suricata e os honeypots representam o lado Blue Team, que observa e reage aos ataques lançados pelo próprio autor contra o seu ambiente controlado.

Filosofia de construção: configurar, depois testar, depois documentar

Este projeto seguiu deliberadamente três fases separadas, em vez de testar cada componente imediatamente após instalá-lo:

1. **Configurar** — instalar e ligar entre si todos os componentes da arquitetura, um a um, confirmando apenas que cada peça arranca e comunica com as vizinhas.
2. **Testar** — só depois de tudo estar de pé, correr os cenários reais de ataque/deteção de ponta a ponta (ex: atacar uma máquina vulnerável e confirmar que o alerta chega ao SIEM e depois ao Telegram).
3. **Documentar** — escrever este manual com o sistema completo e testado à frente, em vez de descrever um alvo em movimento.

Esta separação evita um erro comum em projetos de laboratório: testar cada peça isoladamente à medida que é montada, e nunca chegar a validar se o sistema funciona como um todo. Alguns testes finais (ataques reais, validação da VPN a partir do exterior, o módulo de análise de dispositivos móveis) ainda estavam pendentes no momento em que este manual foi escrito — estão descritos no capítulo final como trabalho planeado.

Arquitetura em blocos

A figura seguinte resume os grandes blocos do sistema e como se relacionam entre si. Os capítulos seguintes detalham cada bloco individualmente — o hardware primeiro, depois o Proxmox como fundação de virtualização, depois cada serviço pela ordem em que foi construído.

Diagrama de arquitetura geral

Nota sobre os endereços IP usados neste manual

Este laboratório corre numa rede doméstica real, com endereços IP reais. Como este documento é publicado abertamente, **todos os endereços IP, domínios e identificadores de rede foram substituídos por valores fictícios**, mantidos consistentes ao longo de todo o manual e destacados visualmente **assim** sempre que aparecem, para que fique sempre claro que são exemplos ilustrativos e não os endereços reais do autor.

Como este manual está organizado

Os capítulos seguem a mesma ordem em que o sistema foi efetivamente construído: primeiro o inventário de equipamento físico (Capítulo 2), depois o Proxmox como camada de virtualização de base (Capítulo 3), depois a rede (Capítulo 4), e a partir daí cada serviço pela ordem em que foi adicionado — AdGuard, Wazuh/Suricata, honeypots, alvos vulneráveis, VPN, alertas de dispositivos, o backend de IA, e a integração com Hack The Box. O Capítulo 13 reúne, para consulta rápida, todos os erros encontrados ao longo da construção e como foram corrigidos. O Capítulo 14 descreve o que ainda está planeado mas não implementado à data deste manual.

Capítulo 2 — Equipamento e Hardware

Antes de entrar em qualquer configuração, importa conhecer as peças físicas que sustentam o laboratório. Nenhuma delas é hardware especializado de nível empresarial — todo o projeto foi construído com equipamento de consumidor, o que é também parte da mensagem: um SOC funcional não exige orçamento corporativo, exige arquitetura bem pensada.

Visão geral da cadeia física

O tráfego, do ponto de vista físico, percorre esta cadeia: modem/router do operador → router com firmware OpenWRT (o verdadeiro "cérebro" de rede do laboratório) → switch gerido → host Proxmox (onde tudo o resto vive como máquina virtual ou contentor). Paralelamente, existem dois dispositivos "clientes" usados para operar e atacar o laboratório: um PC de secretária com uma máquina virtual Kali Linux, e um tablet Android usado para simular dispositivos desconhecidos e testar acesso remoto.

1. Router do operador (ISP)

Router MEO FiberGateway (WiFi 6 / geração GEN8)

O que é: o equipamento fornecido pelo operador de telecomunicações, que termina a ligação de fibra e faz a primeira tradução de endereços (NAT) entre a Internet pública e a rede de casa.

Papel no laboratório: este router é mantido praticamente intocado — só foi necessário abrir uma regra de encaminhamento de porta (port-forward) para permitir que o tráfego VPN, vindo de fora, chegue até ao router interno. Todo o resto da inteligência de rede (firewall, DHCP, DNS, segmentação WiFi) foi deliberadamente movido para um segundo router, mais abaixo na cadeia, porque o firmware de fábrica de routers de operador é normalmente limitado e fechado.

Tipo: Gateway ISP (modem+router combinado) **WiFi:** WiFi 6

Gestão: interface web própria do operador

Credenciais: impressas em etiqueta física no equipamento (não são as genéricas "admin/admin")

2. Router principal — OpenWRT

Xiaomi Redmi Router AX6S, com firmware OpenWRT

O que é: um router doméstico de gama média, com hardware baseado num chipset MediaTek, cujo firmware de fábrica foi substituído pelo **OpenWRT** — um sistema operativo de router open-source, usado extensivamente na indústria e em investigação de redes, que expõe controlo total sobre firewall, encaminhamento, DHCP, DNS e WiFi através de uma interface web (LuCI) e de linha de comandos completa (é, no fundo, um Linux embebido).

Porque substituir o firmware de fábrica: o firmware original de um router de consumidor tipicamente não permite regras de firewall detalhadas, múltiplas redes WiFi isoladas entre si, encaminhamento estático entre sub-redes, ou scripts personalizados a correr em resposta a eventos de rede. O OpenWRT remove essas limitações — e é precisamente isso que este laboratório precisa: duas redes WiFi com políticas diferentes, encaminhamento dedicado para a rede de máquinas vulneráveis, e um script próprio de deteção de dispositivos desconhecidos (detalhado no Capítulo 10).

Papel no laboratório: é o verdadeiro gateway e firewall da rede de casa — todas as regras de segmentação, o encaminhamento da VPN, e o DHCP/DNS internos vivem aqui.

Modelo: Xiaomi Redmi Router AX6S **Arquitetura:** ARMv8

Plataforma: MediaTek MT7622 **Firmware:** OpenWRT (branch 25.x / LuCI 26.x)

Kernel: Linux 6.12

3. Switch gerido

Hisource Hi-S21-8G

O que é: um switch Ethernet de 8 portas Gigabit, "gerido" — ou seja, ao contrário de um switch doméstico simples que apenas liga cabos entre si sem inteligência nenhuma, este permite configurar funcionalidades como VLANs (redes lógicas separadas sobre o mesmo cabo físico) e espelhamento de portas (copiar o tráfego de uma porta para outra, para fins de análise).

Papel no laboratório e uma limitação real encontrada: a ideia inicial era usar o espelhamento de portas do próprio switch para copiar todo o tráfego da rede para a máquina do Wazuh/Suricata analisar. Na prática, isso não foi possível de forma direta porque o host Proxmox só tem **uma única porta de rede física** ligada ao switch — ou seja, não há uma porta física "extra" livre no Proxmox para receber o espelhamento. A solução encontrada foi fazer o espelhamento em software, dentro do próprio Proxmox, copiando o tráfego da sua bridge de rede interna diretamente para a interface virtual da máquina do Wazuh (ver Capítulo 6). O switch em si continua a fazer o que qualquer switch gerido faz de base: interligar todos os cabos da rede física.

Modelo: Hisource Hi-S21-8G **Portas:** 8× Gigabit Ethernet

Tipo: switch gerido (suporta VLAN e espelhamento de porta)

4. Adaptadores Powerline

Par de TP-Link TL-PA7017P

O que é: adaptadores que transportam sinal de rede Ethernet através da própria instalação elétrica de casa (technology PLC — Power Line Communication), evitando ter de passar cabos de rede fisicamente entre divisões.

Papel no laboratório: ligação física entre a divisão onde está o router/switch principal e a divisão onde está o host Proxmox, sem obras.

Modelo: TP-Link TL-PA7017P (par) **Tecnologia:** HomePlug AV2 (powerline)

5. Host de virtualização — Proxmox

Dell OptiPlex 5040 SFF

O que é: um PC de secretária compacto (fator de forma "Small Form Factor"), de gama empresarial, originalmente vendido como posto de trabalho de escritório — não é hardware de servidor dedicado, mas é mais do que suficiente para correr um hipervisor com várias VMs/contentores leves.

Papel no laboratório: é a *base física de tudo* — corre o Proxmox VE (detalhado no Capítulo 3), que por sua vez aloja todas as máquinas virtuais e contentores que compõem o resto do laboratório (Wazuh, AdGuard, honeypots, alvos vulneráveis, servidor VPN, backend de IA).

Modelo: Dell OptiPlex 5040 SFF **CPU:** Intel Core i5-6500 **RAM:** 16 GB

Armazenamento: 256 GB inicial, expandido para 512 GB SSD

6. PC de ataque (Red Team)

PC pessoal com VMware Workstation Pro

O que é: o computador pessoal do autor, usado no dia a dia, sobre o qual corre o VMware Workstation Pro — um hipervisor de desktop (diferente do Proxmox, que é um hipervisor de servidor) — para alojar uma máquina virtual Kali Linux dedicada a ataques.

Porque uma VM separada em vez de dentro do próprio Proxmox: manter a máquina de ataque fora do Proxmox evita misturar o papel de "quem ataca" com "quem hospeda os alvos e as defesas" na mesma infraestrutura, e aproveita hardware com placa gráfica dedicada que o servidor não tem. Chegou a ser considerada a alternativa de instalar Kali diretamente via WSL2 no Windows deste mesmo PC, mas foi abandonada a favor de reutilizar a VM VMware já existente, evitando instalar software adicional sem necessidade.

CPU: AMD Ryzen 7 5800X3D (8 núcleos, 3.40 GHz) **RAM:** 24 GB @ 2400 MT/s

GPU: AMD Radeon RX 7800 XT, 16 GB **Armazenamento:** ~1.13 TB total

Hipervisor: VMware Workstation Pro (VM Kali Linux)

Uma dificuldade real: rede NAT do VMware

O adaptador de rede da VM Kali esteve inicialmente em modo **NAT** (o modo por defeito do VMware), o que causava desconexões frequentes da VPN — o NAT virtual do VMware fecha sessões UDP inativas muito rapidamente, e a VPN usa UDP. A correção foi mudar o adaptador para modo **Bridged**, colocando a VM diretamente na rede física de casa com IP próprio, sem NAT nenhum pelo meio. Este problema e a sua resolução estão detalhados no Capítulo 9.

7. Tablet Android (dispositivo de teste)

Lenovo Tab M10 HD

O que é: um tablet Android de gama de entrada.

Papel no laboratório: desempenha dois papéis distintos. Primeiro, como "dispositivo desconhecido" de teste, para validar o sistema de alerta de novos dispositivos na rede (Capítulo 10) — antes de ser adicionado à lista de dispositivos conhecidos, simula exatamente o cenário que esse sistema foi construído para detetar. Segundo, como cliente VPN móvel, com o seu próprio certificado individual, para testar acesso remoto a partir de um sistema operativo diferente (Android, por oposição ao Linux da VM Kali).

Modelo: Lenovo Tab M10 HD **Sistema:** Android 11

Nota

Um Flipper Zero também pertence ao equipamento pessoal do autor, mas não está, a esta data, integrado no SENTINEL LAB — fica reservado para uma fase futura do projeto e não é mencionado nos capítulos seguintes.

Capítulo 3 — Proxmox VE: a Base de Tudo

Virtualização: o conceito antes da ferramenta

Antes do Proxmox, é preciso entender o problema que a virtualização resolve. Sem ela, cada serviço deste laboratório — o SIEM, o servidor de VPN, o DNS, os honeypots, os alvos vulneráveis — exigiria o seu próprio computador físico dedicado. Isso seria impraticável em casa, tanto em custo como em espaço. A **virtualização** resolve isto ao permitir que um único computador físico (o *host*) corra vários sistemas operativos independentes em simultâneo, cada um a pensar que tem a máquina só para si, através de uma camada de software chamada **hipervisor**.

Existem duas formas principais de fazer isto, e este laboratório usa as duas:

- **Máquina Virtual (VM) completa** — emula um computador inteiro, incluindo o seu próprio kernel (núcleo do sistema operativo). É mais pesada em recursos, mas totalmente isolada e capaz de correr qualquer sistema operativo, incluindo versões antigas ou não convencionais de Linux, ou até Windows.
- **Contentor (LXC/CT)** — partilha o kernel do host, isolando apenas o espaço de utilizador (ficheiros, processos, rede). É muito mais leve e rápido a arrancar, mas está limitado a sistemas operativos compatíveis com o kernel do host (neste caso, Linux).

A escolha entre VM e contentor, para cada serviço deste laboratório, não foi arbitrária — segue uma regra simples: sempre que um serviço precisa de um kernel próprio, específico ou pouco convencional (as máquinas vulneráveis Metasploitable2 e Kioptrix, por exemplo, trazem os seus próprios kernels antigos, incompatíveis com o host), usa-se VM completa. Sempre que um serviço é apenas uma aplicação Linux normal (um servidor DNS, um servidor VPN, uma aplicação web), usa-se um contentor, mais leve e mais rápido.

Porque o Proxmox VE, e não outra alternativa

O Proxmox VE (Virtual Environment) é uma plataforma de virtualização open-source, construída sobre KVM (para VMs completas) e LXC (para contentores), com uma interface web de gestão incluída. Foi escolhido, em vez de alternativas como o VMware ESXi ou o Hyper-V, por três razões concretas: é gratuito e de código aberto (sem

licenciamento para uso pessoal), suporta nativamente os dois modelos de virtualização (VM e contentor) na mesma interface, e tem uma comunidade e documentação muito ativas — relevante quando se está a aprender e a resolver problemas sozinho.

Instalação

O Proxmox foi instalado a partir da imagem ISO oficial, diretamente no Dell OptiPlex 5040 (Capítulo 2), como sistema operativo único da máquina (instalação "bare-metal" — o Proxmox não corre por cima de outro sistema operativo, ele próprio é o sistema operativo do host). Durante a instalação, foram definidos:

- O disco de armazenamento local, formatado com LVM (Logical Volume Manager), que permite redimensionar e gerir o espaço de disco de forma flexível entre as várias VMs/contentores — foi este armazenamento (`local-lvm`) que mais tarde permitiu expandir de 256 GB para 512 GB sem reinstalar nada.
- O endereço IP fixo do host na rede interna, para que a interface de gestão web (acessível na porta 8006) esteja sempre no mesmo endereço.
- A bridge de rede principal, `vmbr0` — uma "ponte" virtual que liga a única porta de rede física do host a todas as VMs/contentores que precisam de estar na rede normal de casa.

Uma segunda bridge, isolada: `vmbr1`

Para a rede de máquinas vulneráveis (Metasploitable2, Kioptrix, DVWA), foi criada uma segunda bridge, `vmbr1`, que **não está ligada a nenhuma porta de rede física** — existe só dentro do próprio Proxmox. Isto cria uma rede verdadeiramente isolada: as máquinas ligadas a ela só se veem entre si e ao próprio Proxmox (que atua como gateway), e não têm, por defeito, qualquer saída para a rede de casa ou para a Internet. O acesso a esta rede a partir de fora (para a atacar deliberadamente) só é possível através da VPN, com encaminhamento explícito, como descrito no Capítulo 9.

Nota sobre os comandos desta secção

Tal como no capítulo anterior, os blocos abaixo são o **procedimento-padrão equivalente** à configuração final confirmada (via `qm list / pct list / pct config`), e não uma transcrição literal e cronológica de cada sessão original.

As duas bridges, em `/etc/network/interfaces`

```
auto lo
iface lo inet loopback

# Bridge principal – liga a NIC física do host à rede de casa (SENTINEL)
auto vmbr0
iface vmbr0 inet static
    address 192.168.50.100 /24
    gateway 192.168.50.1
    bridge-ports enp0s25
    bridge-stp off
    bridge-fd 0

# Bridge isolada – sem bridge-ports nenhuma; existe só dentro do host
auto vmbr1
iface vmbr1 inet static
    address 10.10.99.1 /24
    bridge-ports none
    bridge-stp off
    bridge-fd 0
```

A diferença essencial está na linha `bridge-ports` : a `vmbr0` aponta para a interface física real (`enp0s25`), a `vmbr1` não aponta para nenhuma — é essa ausência que torna a rede isolada verdadeiramente isolada ao nível da própria bridge, e não apenas por regras de firewall.

Criar uma VM completa (exemplo: um dos alvos vulneráveis)

```
qm create 112 \
  --name Kioptrix-L1 \
  --memory 512 \
  --cores 1 \
  --net0 virtio,bridge=vmbr1 \
  --ide0 local-lvm:8,import-from=/var/lib/vz/template/iso/kioptrix-l1.iso
qm start 112
```

Criar um contentor LXC (exemplo: um serviço leve)

```
pct create 108 local:vztmpl/ubuntu-22.04-standard_amd64.tar.zst \
  --hostname adguard \
  --memory 512 \
  --cores 1 \
  --net0 name=eth0,bridge=vmbr0,ip=192.168.50.108/24,gw=192.168.50.1 \
  --unprivileged 1 \
  --features nesting=1
pct start 108
```

Todos os contentores deste laboratório foram criados como **unprivileged** (o modo mais restrito, em que o "root" dentro do contentor não corresponde ao root real do host) — uma camada extra de isolamento, importante sobretudo para o honeypot e para os alvos vulneráveis, cujo propósito é justamente correr código não confiável.

Inventário completo: o que corre no Proxmox

A tabela seguinte é o inventário completo e atualizado de tudo o que corre sobre este host, obtido diretamente através dos comandos `qm list` (máquinas virtuais) e `pct list` (contentores) executados no próprio Proxmox — a forma mais fiável de saber exatamente o que existe, sem depender de memória ou de notas antigas.

ID	Nome	Tipo	Papel	Capítulo
106	Wazuh	VM	SIEM/XDR + Suricata (IDS)	6
111	metasploitable2	VM	Alvo vulnerável (rede isolada)	8
112	Kioptrix-L1	VM	Alvo vulnerável (rede isolada)	8
101	sentinel	CT	Backend de IA (FastAPI)	11
105	vpnlab	CT	Servidor OpenVPN + PKI	9
107	honeypot	CT	Cowrie + OpenCanary	7
108	adguard	CT	AdGuard Home (DNS)	5
113	CT113 (DVWA)	CT	Alvo vulnerável — DVWA	8

Um alvo "perdido": o caso do CT113

Durante a preparação deste manual, ao rever o inventário, um dos contentores (ID 113) aparecia apenas com o nome genérico "CT113" — sem qualquer registo do que continha. Isto aconteceu porque, na altura em que foi criado (uma sessão de trabalho anterior), o hostname nunca chegou a ser personalizado. Em vez de assumir o que seria, a verificação foi feita da forma correta — sem adivinhar — usando `pct config 113` para confirmar a que rede estava ligado, e depois `pct exec 113 -- ls -la /var/www/html` para inspecionar o seu conteúdo real sem necessidade de sequer abrir uma sessão interativa dentro do contentor. O conteúdo (estrutura de ficheiros do DVWA) confirmou tratar-se do alvo vulnerável DVWA, que estava planeado desde o início mas cujo acesso se tinha perdido. Este episódio está detalhado no Capítulo 13 (lista de erros) e serve de lição prática: **o inventário vivo do próprio hipervisor (qm list / pct list) é sempre mais fiável do que notas ou memória**, especialmente num laboratório que cresce ao longo de várias sessões de trabalho.

Snapshots: o "botão de repor" dos alvos de treino

Para as máquinas usadas como alvo de ataques (Kioptrix, e futuramente outras), o Proxmox permite criar um **snapshot** — uma fotografia completa do estado do disco e da memória num dado momento, para a qual se pode voltar instantaneamente. Neste laboratório, cada alvo de treino tem um snapshot chamado `clean-state`, tirado imediatamente após a instalação e antes de qualquer ataque. Isto permite atacar a máquina livremente (incluindo danificá-la irreversivelmente, se for esse o resultado do ataque) e, a seguir, repor o estado limpo em segundos, sem ter de reinstalar nada — essencial para poder repetir exercícios de ataque múltiplas vezes.

Capítulo 4 — Rede: Switch, Router e Segmentação WiFi

Porque segmentar uma rede doméstica

A configuração mais comum de uma rede doméstica é "plana": um único router, uma única rede WiFi, e todos os dispositivos — computadores, telemóveis, impressoras, colunas inteligentes — no mesmo espaço lógico, capazes de se contactar diretamente entre si. Isto é conveniente, mas é também a razão pela qual um único dispositivo comprometido (um telemóvel com uma aplicação maliciosa, por exemplo) pode servir de trampolim para atacar todos os outros: nada o impede de tentar.

Segmentação de rede é o princípio de dividir a rede em zonas lógicas separadas, com regras explícitas sobre que zona pode falar com que zona. Neste laboratório, a segmentação foi usada com dois objetivos diferentes e complementares: proteger a rede principal de trabalho, e criar deliberadamente uma "zona de quarentena" para observar dispositivos desconhecidos sem risco.

As redes deste laboratório, de forma resumida

- **Rede do operador (MEO)** — fora do âmbito desta configuração; é apenas o ponto de entrada da Internet.
- **SENTINEL** — a rede principal, de confiança, onde vivem o Proxmox e todos os serviços. Ligação WiFi na banda de 5 GHz.
- **SENTINELAB** — rede isolada, na banda de 2.4 GHz, sem acesso à Internet e sem qualquer via de acesso à rede SENTINEL. Destina-se a dispositivos desconhecidos ou de teste, cujo tráfego se quer observar (via AdGuard e, mais tarde, Wireshark) sem que consigam alcançar nada sensível.

O switch: interligação física, sem configuração ativa

O switch Hsource Hi-S21-8G (Capítulo 2) é anunciado como "gerido", ou seja, capaz de VLANs e espelhamento de portas. Na prática, **nenhuma dessas funcionalidades chegou a ser configurada** — o switch serve, até hoje, apenas para interligar fisicamente os cabos de rede entre o router, o Proxmox e os adaptadores powerline.

Limitação conhecida, ainda não resolvida

A interface de gestão do switch (esperada no endereço **192.168.50.2**) nunca foi alcançada com sucesso a partir do browser — as tentativas de acesso não obtiveram resposta. Não foi ainda diagnosticada a causa (pode ser um endereço IP diferente do assumido, uma porta de gestão dedicada, ou uma funcionalidade que precisa de ser ativada por outro meio). Por esta razão, todas as funcionalidades de gestão do switch (VLANs, espelhamento de porta) permanecem por explorar, e a decisão de fazer o espelhamento de tráfego em software, diretamente no Proxmox (Capítulo 6), não foi por preferência de desenho, mas também consequência direta desta limitação. Fica registado como item a investigar numa fase futura.

O router OpenWRT: o verdadeiro cérebro da rede

Como referido no Capítulo 2, o firmware de fábrica do router foi substituído por OpenWRT precisamente para obter controlo total sobre firewall, DHCP, DNS e WiFi. É aqui que a segmentação SENTINEL/SENTINELAB é implementada, através de três mecanismos que trabalham em conjunto: interfaces de rede separadas, zonas de firewall, e redes WiFi (SSIDs) dedicadas a cada interface.

Nota sobre os comandos desta secção

Os comandos abaixo reconstituem, através da interface UCI (a ferramenta de configuração de linha de comandos do OpenWRT), a configuração final que sabemos que está em vigor — confirmada e testada — mas não são uma transcrição literal e cronológica de cada passo original. Servem para que qualquer pessoa possa reproduzir a mesma arquitetura.

Interfaces e WiFi

```
# Duas interfaces de rede lógicas, uma por cada zona
uci set network.sentinel=interface
uci set network.sentinel.proto='static'
uci set network.sentinel.ipaddr=' 192.168.50.1 '
uci set network.sentinel.netmask='255.255.255.0'

uci set network.sentinelab=interface
uci set network.sentinelab.proto='static'
uci set network.sentinelab.ipaddr=' 192.168.60.1 '
uci set network.sentinelab.netmask='255.255.255.0'

# Rede WiFi "SENTINEL", 5 GHz (radio1), associada à interface sentinel
uci set wireless.sentinel_wifi=wifi-iface
uci set wireless.sentinel_wifi.device='radio1'
uci set wireless.sentinel_wifi.network='sentinel'
uci set wireless.sentinel_wifi.mode='ap'
uci set wireless.sentinel_wifi.ssid='SENTINEL'
uci set wireless.sentinel_wifi.encryption='psk2'

# Rede WiFi "SENTINELAB", 2.4 GHz (radio0), associada à interface sentinelab
uci set wireless.sentinelab_wifi=wifi-iface
uci set wireless.sentinelab_wifi.device='radio0'
uci set wireless.sentinelab_wifi.network='sentinelab'
uci set wireless.sentinelab_wifi.mode='ap'
uci set wireless.sentinelab_wifi.ssid='SENTINELAB'
uci set wireless.sentinelab_wifi.encryption='psk2'

uci commit wireless
uci commit network
/etc/init.d/network restart
```

Isolamento por firewall (a parte que realmente importa)

Ter interfaces separadas não isola nada por si só — sem uma regra de firewall explícita, o router encaminharia tráfego entre as duas zonas livremente. O isolamento real vem de colocar cada interface numa **zona de firewall** distinta, e não permitir encaminhamento (*forwarding*) da zona de quarentena para a zona de confiança:

```
uci set firewall.zone_sentinelab=zone
uci set firewall.zone_sentinelab.name='sentinelab'
uci set firewall.zone_sentinelab.network='sentinelab'
uci set firewall.zone_sentinelab.input='REJECT'
uci set firewall.zone_sentinelab.output='ACCEPT'
uci set firewall.zone_sentinelab.forward='REJECT' # <- chave: sem forward para a
LAN principal

uci commit firewall
/etc/init.d/firewall restart
```

Com `forward='REJECT'` nesta zona (e sem nenhuma regra explícita de excepção a permitir o contrário), um dispositivo ligado à SENTINELAB pode obter DHCP e resolver DNS, mas não consegue iniciar ligação nenhuma à rede SENTINEL nem à Internet — exactamente o comportamento pretendido para uma zona de observação.

DHCP e DNS: a ligação ao AdGuard

Ambas as redes recebem o seu endereço IP via DHCP servido pelo próprio OpenWRT, mas o servidor de DNS entregue a cada dispositivo é o **AdGuard Home** (Capítulo 5), e não o do OpenWRT ou da operadora — é isto que dá ao AdGuard visibilidade sobre tudo o que cada rede tenta contactar:

```
uci set dhcp.sentinel.dhcp_option='6,192.168.50.108 '  
uci set dhcp.sentinelab.dhcp_option='6,192.168.50.108 '  
uci commit dhcp  
/etc/init.d/dnsmasq restart
```

A descoberta da dupla-NAT e o acesso remoto por VPN

Ao preparar o acesso remoto por VPN (detalhado a fundo no Capítulo 9), foi descoberta uma particularidade da instalação: o router OpenWRT não está ligado directamente à Internet — o seu WAN liga a um *segundo* router, o da própria operadora (MEO), que por sua vez é que está ligado à fibra. Ou seja, existem **duas camadas de NAT (Network Address Translation)** em série: a rede de casa → OpenWRT (1ª tradução de endereços) → router MEO (2ª tradução de endereços) → Internet.

Isto tem uma consequência prática direta: para que tráfego originado na Internet chegue ao servidor VPN dentro de casa, é preciso abrir a mesma porta em **ambos** os routers — um único encaminhamento de porta não basta.

Encaminhamento de porta no router OpenWRT

```
uci add firewall redirect
uci set firewall.@redirect[-1].name='OpenVPN-to-vpnlab'
uci set firewall.@redirect[-1].src='wan'
uci set firewall.@redirect[-1].dest='sentinel'
uci set firewall.@redirect[-1].dest_ip=' 192.168.50.105 '
uci set firewall.@redirect[-1].dest_port='1194'
uci set firewall.@redirect[-1].proto='udp'
uci set firewall.@redirect[-1].target='DNAT'
uci commit firewall
/etc/init.d/firewall restart
```

Confirmação real, tal como verificada durante a construção deste laboratório:

```
root@OpenWrt:~# uci show firewall | grep -A9 "OpenVPN-to-vpnlab"
firewall.@redirect[3]=redirect
firewall.@redirect[3].name='OpenVPN-to-vpnlab'
firewall.@redirect[3].src='wan'
firewall.@redirect[3].dest_ip=' 192.168.50.105 '
firewall.@redirect[3].dest_port='1194'
firewall.@redirect[3].proto='udp'
firewall.@redirect[3].target='DNAT'
```

Encaminhamento de porta no router MEO (segunda camada)

No router da operadora, o encaminhamento tem de apontar não para o servidor VPN diretamente, mas para o endereço que o router OpenWRT ocupa *dentro* da rede da MEO (o seu IP de WAN, visto do lado de fora):

Dificuldade encontrada: credenciais de acesso ao router MEO

O login por defeito para este modelo (WiFi 6 / geração GEN8) não é o clássico `meo/meo` nem `admin/admin` usado em modelos mais antigos — ambos foram tentados sem sucesso. A credencial correta está impressa numa etiqueta física, colada no próprio equipamento. Depois de autenticado, o caminho no menu também não é óbvio: a regra fica em **Segurança → Acesso** (e não em "Segurança → Firewall e DMZ", onde se tentou primeiro).

Campo	Valor
Serviço	OpenVpn
IP do servidor	192.168.100.216
Protocolo	UDP
Porta externa	1194
Porta interna	1194

Segunda camada de defesa: bloquear acesso externo ao Proxmox

Depois de confirmar que nenhum encaminhamento expõe a interface de gestão do Proxmox (porta 8006) à Internet, foi ainda adicionada uma regra de firewall explícita no OpenWRT, como proteção adicional contra um encaminhamento futuro criado por engano:

```
uci add firewall rule
uci set firewall.@rule[-1].name='Block-WAN-to-Proxmox-8006'
uci set firewall.@rule[-1].src='wan'
uci set firewall.@rule[-1].dest_ip='192.168.50.100'
uci set firewall.@rule[-1].dest_port='8006'
uci set firewall.@rule[-1].target='REJECT'
uci commit firewall
/etc/init.d/firewall restart
```

Deteção de dispositivos desconhecidos

Ainda no OpenWRT corre um script próprio, `check_new_devices.sh`, que vigia a tabela ARP da rede SENTINEL e envia um alerta (por Telegram) sempre que aparece um endereço MAC que não conste de uma lista de dispositivos conhecidos. Como este script está intimamente ligado à resposta a incidentes e não apenas à topologia de rede, é descrito com detalhe no Capítulo 10, em conjunto com o mecanismo de resposta automática do Wazuh.

Capítulo 5 — AdGuard Home: Filtragem e Visibilidade de DNS

Como funciona o DNS, resumidamente

Sempre que um dispositivo quer aceder a um site (por exemplo, `exemplo.com`), não sabe naturalmente onde esse site está — só sabe endereços IP, não nomes. O **DNS (Domain Name System)** é o sistema que traduz nomes legíveis por humanos em endereços IP. O processo, chamado resolução de nomes, funciona normalmente assim: o dispositivo pergunta "qual é o IP de exemplo.com?" a um servidor DNS configurado na rede; esse servidor responde, o dispositivo guarda a resposta em cache por um tempo, e só depois é feita a ligação real ao site.

Isto é importante em segurança por uma razão muitas vezes esquecida: mesmo que o tráfego seguinte seja todo cifrado por HTTPS (o que impede ver o *conteúdo* da comunicação), a pergunta DNS que o precede tipicamente não é — quem controla o servidor DNS de uma rede consegue ver, no mínimo, todos os domínios que cada dispositivo tentou contactar, mesmo sem conseguir ver o que foi trocado depois.

O que é o AdGuard Home, concretamente

O AdGuard Home é um servidor DNS próprio, autoalojado, que substitui esse servidor "por defeito". Isto dá duas capacidades: **bloqueio** (responder "este domínio não existe" para domínios conhecidos por anúncios, rastreamento ou malware, impedindo a ligação antes de acontecer) e **visibilidade** (um registo completo — data, hora, dispositivo, domínio — de tudo o que cada máquina tentou contactar).

Num SOC, a primeira pergunta sobre um dispositivo suspeito costuma ser "com quem é que ele está a falar?". O AdGuard responde exatamente a isso, sem decifrar tráfego nenhum.

Instalação

O AdGuard Home corre num contentor LXC dedicado (CT108, Capítulo 3), instalado a partir do script oficial de instalação, que descarrega o binário, cria o serviço systemd e expõe a interface de administração na porta 3000 (primeira configuração) e depois na porta 80 (uso normal):

```
curl -s -S -L https://raw.githubusercontent.com/AdguardTeam/AdGuardHome/master/scripts/install.sh | sh -s -- -v  
systemctl status AdGuardHome
```

Dois perfis de cliente, com propósitos opostos

O AdGuard permite definir definições diferentes por "cliente" (identificado por IP, intervalo CIDR, ou MAC). Neste laboratório, cada uma das duas redes WiFi (Capítulo 4) tem o seu próprio perfil, com filosofias deliberadamente opostas.

Perfil "SENTINELAB" — observar sem interferir

Identificador: CIDR **192.168.60.0/24**. Todas as opções de proteção estão **desativadas** (sem bloqueio de domínios, sem safe browsing, sem controlo parental, sem pesquisa segura). O motivo é simples: esta é a rede de quarentena/análise, onde o objetivo é ver o comportamento real e completo de um dispositivo desconhecido — incluindo os domínios "maus" que tentaria contactar. Bloquear esses domínios esconderia exatamente o sinal que se quer observar. O registo de consultas e as estatísticas ficam ativos (não ignorados), porque é esse registo que constitui a análise.

Perfil "SENTINEL" — proteção real do dia a dia

Identificador: CIDR **192.168.50.0/24**. Aqui o objetivo é o oposto: esta é a rede de trabalho principal, por isso a proteção fica **ativa** — bloqueio de domínios via listas de filtros, segurança de navegação (safe browsing) ativa. Controlo parental e pesquisa segura ficam desativados (não aplicável ao caso de uso). O registo de consultas e estatísticas também fica ativo, para manter visibilidade total mesmo na rede de confiança.

Definição	SENTINEL (principal)	SENTINELAB (quarentena)
Bloqueio de domínios (filtros/ hosts)	Ativo	Desativado
Segurança de navegação	Ativo	Desativado
Controlo parental	Desativado	Desativado
Pesquisa segura	Desativado	Desativado
Registo de consultas / estatísticas	Ativo	Ativo

Porque isto importa para a deteção de dispositivos desconhecidos

Combinado com o script de deteção de dispositivos novos do OpenWRT (Capítulo 10), este desenho significa que qualquer dispositivo desconhecido que apareça na rede pode ser imediatamente movido (ou já nasce) na SENTINELAB, onde todo o seu tráfego DNS fica visível e nada é bloqueado — maximizando a informação disponível para decidir se é uma ameaça, antes de qualquer ação de bloqueio ser tomada.

Capítulo 6 — Wazuh (SIEM/XDR) e Suricata (IDS)

Dois conceitos que se confundem, mas são diferentes

Um **IDS (Intrusion Detection System)** — Suricata, aqui — inspeciona o tráfego de rede pacote a pacote, à procura de padrões conhecidos de ataque (assinaturas), e gera um alerta quando encontra uma correspondência. Não sabe nada sobre o resto do sistema — só vê pacotes.

Um **SIEM (Security Information and Event Management)** — Wazuh, aqui — faz o inverso: não olha para pacotes de rede diretamente, mas recolhe e centraliza *eventos* de múltiplas fontes (logs de sistema, alertas de outras ferramentas, integridade de ficheiros, configuração de segurança) e correlaciona-os, para que um humano (ou uma automação) consiga ver o quadro completo em vez de dezenas de fontes soltas. A versão moderna do Wazuh acrescenta capacidades de **XDR (Extended Detection and Response)** — não só agregar, mas também responder automaticamente (ver Capítulo 10).

Neste laboratório, o Suricata corre *dentro* da própria VM do Wazuh, e os seus alertas são lidos pelo Wazuh como mais uma fonte de eventos — o IDS gera o sinal de rede, o SIEM absorve-o, junta-o a tudo o resto, e decide se e como reagir.

Instalação do Wazuh

O Wazuh corre numa VM completa dedicada (VMID 106, Capítulo 3) — e não num contentor — porque a stack Wazuh (manager, indexer baseado em OpenSearch, dashboard) tem requisitos de `sysctl` e de kernel que não se garantem de forma fiável num contentor LXC não privilegiado. A instalação usou o script "all-in-one" oficial:

```
curl -s0 https://packages.wazuh.com/4.x/wazuh-install.sh  
bash wazuh-install.sh -a
```

Reforço de segurança: password do dashboard

A password de instalação gerada automaticamente para o utilizador `admin` do Wazuh Dashboard foi substituída por uma escolhida pelo autor, usando a ferramenta oficial de gestão de passwords do indexer:

```
/usr/share/wazuh-indexer/plugins/opensearch-security/tools/wazuh-passwords-tool.sh -u admin -p '<nova-password>'
```

Não foi criado um utilizador adicional nomeado — a ferramenta só permite alterar a password de utilizadores predefinidos (`admin`, `kibanaserver`, `wazuh`, `wazuh-wui`); criar um utilizador novo exigiria editar manualmente ficheiros YAML de segurança internos, com risco real de bloquear o próprio acesso ao dashboard — considerado um risco desnecessário para o benefício.

Suricata: regras, atualização e integração

O Suricata foi instalado na mesma VM, a escutar na sua interface de rede. Depois da instalação de base, o conjunto de regras foi atualizado com o *Emerging Threats Open* — uma coleção de assinaturas de ataque mantida pela comunidade e gratuita:

```
suricata-update  
systemctl restart suricata
```

Isto trouxe milhares de regras genéricas. Complementarmente, foram escritas **16 regras próprias**, focadas num sinal de risco muito concreto e fácil de justificar: serviços sensíveis expostos que não deveriam estar acessíveis sem cuidado extra — RDP (3389), SMB (445/139), Telnet (23), VNC (5900), FTP (21), bases de dados (MySQL 3306, PostgreSQL 5432, MSSQL 1433, Redis 6379, MongoDB 27017), SNMP (161), e portas classicamente associadas a shells reversas/C2 (4444, 1337, 31337, 6666-6667):

```
# /var/lib/suricata/rules/local.rules (excerto)  
alert tcp any any -> $HOME_NET 3389 (msg:"Tentativa de ligacao RDP detectada"; \  
  flow:to_server; classtype:attempted-recon; sid:1000001; rev:1;)  
alert tcp any any -> $HOME_NET 4444 (msg:"Porta classica de shell reversa (4444)"; \  
  flow:to_server; classtype:trojan-activity; sid:1000010; rev:1;)  
# (+ 14 regras semelhantes, sids 1000002-1000016)
```

A ligação entre Suricata e Wazuh é feita através do ficheiro de eventos do próprio Suricata (`eve.json`, em formato JSON), que o agente Wazuh local lê como uma fonte de log normal:

```
<!-- /var/ossec/etc/ossec.conf, dentro do bloco <ossec_config> -->
<localfile>
  <log_format>json</log_format>
  <location>/var/log/suricata/eve.json</location>
</localfile>
```

Ver o resto da rede: o problema do espelhamento de tráfego

Por defeito, o Suricata só vê o tráfego da própria interface de rede da VM onde corre — ou seja, apenas o tráfego dirigido à própria VM Wazuh, e não o resto da rede. Para que sirva de verdade como sonda de rede, precisa de receber uma **cópia** de todo o tráfego que passa pela rede, não apenas o seu.

A forma clássica de resolver isto é o espelhamento de porta no switch gerido — mas, como descrito no Capítulo 4, a interface de gestão do switch nunca foi alcançada. A alternativa escolhida foi espelhar o tráfego **em software, dentro do próprio Proxmox**, copiando tudo o que entra e sai da bridge principal (`vmbr0`) diretamente para a interface de rede virtual da VM do Wazuh, usando as capacidades de *traffic control* do kernel Linux (`tc`):

```
# No host Proxmox – espelha ingress e egress de vmbr0 para a interface da VM Wazuh
tc qdisc add dev vmbr0 clsact
tc filter add dev vmbr0 ingress matchall action mirred egress mirror dev tap106i0
tc filter add dev vmbr0 egress matchall action mirred egress mirror dev tap106i0
```

Este truque funciona porque a bridge `vmbr0` vê, por definição, todo o tráfego que passa por qualquer porta ligada a ela — replicar essa visão para a interface virtual do Suricata dá-lhe exatamente o mesmo ponto de vista que teria um espelhamento de porta físico no switch.

Uma fragilidade conhecida deste método

A interface virtual referenciada (`tap106i0` no exemplo) corresponde à VM com ID 106. Se essa VM for alguma vez recriada do zero (não apenas reiniciada), o Proxmox pode atribuir-lhe uma interface com nome diferente, e as regras `tc` deixariam de espelhar para o sítio certo — teriam de ser reaplicadas manualmente. Estas regras foram tornadas persistentes a reinícios do host através de linhas `post-up` no bloco da `vmbr0` em `/etc/network/interfaces`, mas essa persistência não protege contra uma recriação da VM.

Teste de validação: tráfego dirigido a outro serviço da rede (não à própria VM Wazuh) foi corretamente capturado e alertado pelo Suricata, confirmando que o espelhamento está mesmo a funcionar e não apenas a ver o seu próprio tráfego local.

Capítulo 7 — Honeypots: Cowrie e OpenCanary

O que é um honeypot, e porque vale a pena ter um

Um **honeypot** é um sistema deliberadamente exposto e deliberadamente falso — finge ser um servidor real (SSH, FTP, uma base de dados), aceita ligações e até credenciais, mas não faz nada de real por trás: só regista tudo o que o atacante tentou. Não protege nada diretamente; o seu valor está em dar **visibilidade antecipada**. Qualquer interação com um honeypot é, por definição, suspeita — nenhum utilizador legítimo tem razão para lá tocar — o que faz dele uma fonte de alertas com uma taxa de falsos positivos praticamente nula, ao contrário de um IDS genérico, que pode gerar ruído a partir de tráfego legítimo mas atípico.

Este laboratório usa dois honeypots complementares, ambos no mesmo contentor (CT107): o **Cowrie**, especializado em SSH/Telnet, e o **OpenCanary**, que simula vários serviços simples em simultâneo (FTP, HTTP, MySQL, SMB).

Cowrie

O Cowrie simula uma sessão SSH completa e interativa — aceita qualquer combinação de utilizador/password (é esse o objetivo: nunca recusar um atacante), e depois simula um sistema de ficheiros e um conjunto de comandos, registando tudo o que é escrito.

Erro grave encontrado: honeypot sem SIEM durante dias

Foi descoberto que o Cowrie esteve a correr desde a sua instalação inicial **sem que o agente Wazuh estivesse instalado no contentor** — ou seja, todos os eventos capturados nunca chegaram ao SIEM. O honeypot estava a "ouvir" corretamente, mas ninguém estava a "ler" o que ele ouvia.

Correção aplicada

Instalação e registo do agente Wazuh no contentor, ligação do ficheiro de log JSON do Cowrie ao agente, e criação de regras próprias no manager para classificar corretamente cada tipo de evento do Cowrie:

```
# Instalar e registar o agente
apt install ./wazuh-agent_4.14.7-1_amd64.deb
/var/ossec/bin/agent-auth -m 192.168.50.243

# ossec.conf do agente – ler o log JSON do Cowrie
<localfile>
  <log_format>json</log_format>
  <location>/home/cowrie/cowrie/var/log/cowrie/cowrie.json</location>
</localfile>
```

```
<!-- local_rules.xml no manager – grupo "cowrie,honeypot," -->
<rule id="100102" level="10">
  <if_group>cowrie</if_group>
  <field name="eventid">cowrie.login.success</field>
  <description>Cowrie: login com sucesso no honeypot SSH</description>
</rule>
<!-- + regras 100100-100106 para connect, login.failed, command.input,
session.closed, catch-all -->
```

Depois desta correção, testado com sucesso: os alertas do Cowrie passaram a aparecer no Wazuh com a descrição correta para cada tipo de evento (ligação, falha de login, login bem-sucedido, comando introduzido, sessão terminada).

OpenCanary

O OpenCanary foi instalado no mesmo contentor, num ambiente Python isolado (*virtual environment*), para contornar um problema de compatibilidade entre versões do `pip` / `setuptools` que impedia a opção oficial `--copyconfig` de funcionar:

```
python3 -m venv /opt/opencanary-env
source /opt/opencanary-env/bin/activate
pip install opencanary
opencanaryd --copyconfig # falhava aqui – contornado gerando o config manualmente
```

Os módulos FTP, HTTP e MySQL funcionaram diretamente, ligando-se às respetivas portas (21, 80, 3306) e simulando os serviços sem necessidade de mais nada instalado.

Dificuldade real: o módulo SMB, e um bug em cadeia difícil de diagnosticar

O módulo SMB do OpenCanary é diferente dos outros: **não implementa o protocolo SMB** — em vez disso, lê o ficheiro de log de auditoria de um servidor Samba real, à procura de acessos. Isto obrigou a instalar Samba a sério (com uma pasta-isco, `[Backups]`, acessível como convidado) e a configurar o módulo `full_audit` do Samba para escrever essas auditorias.

Depois disto, os alertas continuavam a não aparecer. A investigação revelou uma cadeia de três problemas empilhados:

1. Permissões erradas no ficheiro de log de auditoria (tinha de ser `syslog:adm 640`, tal como o `/var/log/syslog` do sistema, e não `root:root 644`).
2. Depois de corrigido isso, o formato do prefixo de auditoria do Samba (`full_audit:prefix`) tinha de ter **exatamente 10 campos** — o parser do OpenCanary para o módulo SMB está escrito à espera de 13 campos no total (10 do prefixo + 3 acrescentados automaticamente pelo próprio Samba). O prefixo original, mais curto, produzia linhas com menos campos do que esperado.
3. E o efeito desse desalinhamento não era um erro visível — era silencioso: uma linha mal formatada causava uma exceção não tratada dentro do gestor de eventos do ficheiro (inotify) do OpenCanary, que **matava permanentemente** esse vigilante de ficheiro, sem qualquer mensagem no log principal do OpenCanary (só visível no log de sistema geral, através da etiqueta do processo).

```
# smb.conf – prefixo com exatamente os 10 campos que o parser espera
full_audit:prefix = %U|%I|%i|%m|%S|%L|%R|%a|%T|%D
full_audit:success = connect disconnect open close read write rename unlink
full_audit:failure = connect
full_audit:facility = local5
full_audit:priority = notice
```

Depois de corrigir o prefixo e reiniciar o serviço, o módulo SMB passou a gerar eventos completos e corretos.

Integração OpenCanary → Wazuh

```
<!-- ossec.conf do agente – mesmo contentor do Cowrie -->
<localfile>
  <log_format>json</log_format>
  <location>/var/tmp/opencanary.log</location>
</localfile>
```

```
<!-- local_rules.xml – grupo "opencanary,honeypot," -->
<rule id="100122" level="10">
  <if_group>opencanary</if_group>
  <field name="logdata.logtype">2000</field>
  <description>OpenCanary: tentativa de login FTP no honeypot</description>
</rule>
<!-- + regras 100120-100127 para HTTP, MySQL, SMB e mensagens de sistema -->
```

Testado end-to-end com sucesso para os quatro serviços: login FTP, login HTTP (POST), login MySQL, e acesso SMB — todos geraram o alerta correto no Wazuh, com a descrição correspondente ao serviço específico.

Capítulo 8 — A Rede Isolada de Alvos Vulneráveis

Porque ter alvos vulneráveis de propósito

Para praticar deteção e resposta, é preciso primeiro haver algo para detetar. Este laboratório mantém uma pequena coleção de sistemas deliberadamente cheios de falhas conhecidas, isolados na rede `10.10.99.0/24` (bridge `vmbr1`, Capítulo 3) — sem qualquer saída para a Internet ou para a rede principal, acessível apenas através da VPN com encaminhamento explícito (Capítulo 9).

Metasploitable2

Uma distribuição Linux (VM 111) construída especificamente para conter o máximo de vulnerabilidades exploráveis possível — serviços antigos e mal configurados por todo o lado (FTP anónimo, bases de dados sem password, versões de serviços com exploits públicos conhecidos). Serve como o alvo "generalista", com múltiplos caminhos de ataque possíveis.

Kioptrix Level 1

Uma máquina de treino descarregada do VulnHub — uma comunidade que distribui imagens de máquinas virtuais deliberadamente vulneráveis, cada uma pensada como um desafio único e progressivo. O Kioptrix Level 1 é geralmente considerado o ponto de entrada clássico para quem começa em pentesting, e foi por isso a primeira escolha, com a intenção de progredir depois para versões mais difíceis da mesma série (1.1, 1.2, 2...).

```
wget https://download.vulnhub.com/kioptrix/Kioptrix_Level_1.rar
# verificação de integridade do download
md5sum Kioptrix_Level_1.rar
```

Erro encontrado: kernel panic ao arrancar

Depois de importado no Proxmox, a VM entrava em *kernel panic* logo no arranque, com a mensagem "No init found". A causa: o disco foi importado por defeito como `sata0`, mas o kernel desta máquina é muito antigo (Red Hat / `nash`, de cerca de 2014) e espera encontrar o seu disco como IDE (`/dev/hda`), não como SATA.

Correção

Recriar o disco da VM como `ide0` em vez de `sata0`. Depois desta alteração, a máquina arrancou normalmente.

```
qm set 112 --ide0 local-lvm:vm-112-disk-0
qm set 112 --delete sata0
```

Um snapshot `clean-state` (Capítulo 3) foi criado imediatamente após confirmar o arranque correto, antes de qualquer tentativa de ataque.

DVWA (Damn Vulnerable Web Application)

O DVWA é uma aplicação web em PHP, também deliberadamente vulnerável, mas ao contrário das duas anteriores — que são sistemas operativos inteiros para explorar por múltiplas vias — foca-se especificamente em vulnerabilidades de aplicações web (SQL injection, XSS, upload de ficheiros, etc.), com um nível de dificuldade ajustável diretamente na aplicação.

Um alvo temporariamente "perdido"

Este é o mesmo episódio já descrito no Capítulo 3: o DVWA existia — no contentor CT113 — mas o seu hostname nunca tinha sido personalizado, e o registo de onde exatamente tinha sido instalado perdeu-se entre sessões de trabalho. Em vez de o recriar do zero (o que teria sido o caminho mais rápido, mas teria destruído trabalho existente sem necessidade), a investigação seguiu o caminho correto: `pct list` no host Proxmox para ver todos os contentores existentes, `pct config 113` para confirmar a que rede estava ligado, e finalmente `pct exec 113 -- ls -la /var/www/html` para inspecionar o seu conteúdo sem necessidade de sequer entrar interativamente no contentor:

```
root@sentinel-lab:~# pct exec 113 -- ls -la /var/www/html
drwxr-xr-x 12 root root 4096 ... .
drwxr-xr-x  6 root root 4096 ... dvwa
drwxr-xr-x  5 root root 4096 ... hackable
drwxr-xr-x 21 root root 4096 ... vulnerabilities
-rw-r--r--  1 root root 4064 ... login.php
-rw-r--r--  1 root root 5552 ... setup.php
```

A presença da estrutura oficial do repositório DVWA confirmou definitivamente o que era. Uma verificação adicional (`ss -tlnp`) confirmou a stack completa a funcionar: Apache na porta 80, MariaDB na porta 3306 (só acessível localmente), SSH na porta 22.

Acesso: através da VPN, no endereço **10.10.99.4**, porta 80. Como em qualquer instalação DVWA, existe uma opção "Create / Reset Database" (em `setup.php`) para (re)inicializar a base de dados do zero, caso as credenciais de administrador se percam.

Lição geral deste capítulo

Todos os três alvos revelaram, de formas diferentes, a mesma lição: o inventário vivo do hipervisor (`qm list` / `pct list` / `pct config` / `pct exec`) é sempre a fonte de verdade mais fiável — mais do que memória, notas antigas, ou suposições sobre o que "deveria" estar lá.

Capítulo 9 — Acesso Remoto Seguro: OpenVPN

O que é uma VPN, e o que a torna "privada" e "virtual"

Uma **VPN (Virtual Private Network)** cria um túnel cifrado entre dois pontos através de uma rede não confiável (a Internet) — tudo o que passa dentro do túnel fica invisível e inalterável para quem estiver a observar o tráfego pelo caminho, e do ponto de vista do dispositivo ligado, é como se estivesse fisicamente ligado à rede remota: recebe um endereço IP dessa rede, e passa a poder alcançar tudo o que essa rede permitir. É "virtual" porque não há nenhum cabo novo — o túnel existe só em software, sobre a ligação à Internet normal — e "privada" porque, embora atravesse uma rede pública, o conteúdo permanece cifrado e inacessível a terceiros.

Neste laboratório, a VPN (OpenVPN, num contentor dedicado, `vpnlab`, CT105) tem um propósito muito concreto: permitir gerir e analisar o laboratório a partir de qualquer lugar — e, para os dispositivos que servem também para atacar (a VM Kali), alcançar a rede isolada de alvos vulneráveis sem a expor ao mundo.

Autenticação por certificado: a lógica por trás da PKI

Em vez de uma simples password partilhada, o OpenVPN aqui usa uma **PKI (Public Key Infrastructure)** baseada em certificados: existe uma autoridade certificadora própria (CA), criada uma única vez, que assina um certificado individual para o servidor e um certificado individual para cada dispositivo cliente. Cada certificado de cliente prova a sua identidade através de criptografia assimétrica (uma chave privada, que nunca sai do dispositivo, e uma chave pública, embutida no certificado assinado pela CA) — o servidor confia em qualquer certificado assinado pela sua própria CA, sem precisar de saber de antemão quantos ou quais dispositivos existirão.

Porque um certificado por dispositivo, e não um único certificado compartilhado

Seria tecnicamente mais simples gerar um único certificado e copiá-lo para todos os dispositivos. A razão para não o fazer é a **revogação**: se um dispositivo for perdido, roubado, ou o seu certificado for exposto por acidente, um certificado individual permite revogar exatamente esse dispositivo, sem afetar mais nenhum. Com um certificado compartilhado, a única opção seria revogar tudo e reemitir novos certificados para todos os dispositivos de uma vez. Esta prática (um certificado por dispositivo) foi adotada como norma para todo o equipamento presente e futuro deste laboratório.

Criação da PKI e do servidor

```
easyrsa init-pki
easyrsa build-ca nopass
easyrsa gen-req server nopass
easyrsa sign-req server server
easyrsa gen-dh
openvpn --genkey secret ta.key # chave adicional de HMAC (tls-auth)
```

```
# /etc/openvpn/server.conf (excerto)
port 1194
proto udp
dev tun
topology subnet
server 10.8.99.0 255.255.255.0
push "route 192.168.50.0 255.255.255.0"
push "route 10.10.99.0 255.255.255.0"
push "dhcp-option DNS 192.168.50.108"
tls-auth ta.key 0
cipher AES-256-CBC
keepalive 5 30
crl-verify /etc/openvpn/easy-rsa/pki/crl.pem
persist-key
persist-tun
```

Incidente de segurança: chave privada exposta no chat, e uma falha estrutural descoberta ao corrigi-lo

Durante a construção, o conteúdo completo de um ficheiro `.ovpn` — incluindo a chave privada RSA e a chave `tls-auth` — foi colado inadvertidamente numa conversa de apoio. Qualquer chave privada exposta desta forma deve ser considerada comprometida, independentemente de alguém a ter realmente usado.

Ao corrigir isto, foi descoberto um problema mais profundo, e mais preocupante: o `server.conf` não tinha a diretiva `crl-verify` configurada. Isto significa que, mesmo depois de revogar o certificado comprometido, **o servidor continuaria a aceitá-lo** — a revogação por si só não bloqueia nada sem esta diretiva, que é a que efetivamente manda o servidor consultar a lista de certificados revogados a cada nova ligação.

Correção em duas partes

```
# 1. Revogar o certificado exposto
easyrsa revoke craveiro
easyrsa gen-crl
systemctl restart openvpn@server

# 2. Corrigir a falha estrutural — adicionar a verificação da lista de
revogação
echo "crl-verify /etc/openvpn/easy-rsa/pki/crl.pem" >> /etc/openvpn/
server.conf
systemctl restart openvpn@server
```

Um novo certificado foi emitido para substituir o revogado. Esta é a razão pela qual, mais tarde, o processo de revogar-e-reemitir se tornou uma operação de rotina bem compreendida (usada de novo, deliberadamente, para adicionar frase-passe ao certificado da VM Kali — ver mais abaixo).

Reparar o encaminhamento de ponta a ponta

Ter certificados válidos não é suficiente — para que um cliente VPN alcance tanto a rede principal como a rede isolada de alvos vulneráveis, o encaminhamento IP tem de estar corretamente ativo em **todos** os pontos do percurso. Foi descoberto que o encaminhamento (`ip_forward`) estava desativado tanto no contentor `vpnlab` como no

próprio host Proxmox — o que sugere que o acesso à rede isolada através da VPN provavelmente nunca tinha funcionado antes desta correção.

```
# Em vpnlab E no host Proxmox
sysctl -w net.ipv4.ip_forward=1
echo "net.ipv4.ip_forward=1" >> /etc/sysctl.conf
```

Persistência com uma particularidade do Proxmox

Tornar isto permanente a reinícios exigiu um mecanismo diferente em cada sítio: no host Proxmox, uma linha `post-up` no bloco de rede em `/etc/network/interfaces` (o mesmo mecanismo usado para as regras de espelhamento do Capítulo 6); no contentor `vpnlab`, foi criado um serviço systemd *oneshot* dedicado (`extra-routes.service`), em vez de editar o ficheiro de rede que o próprio Proxmox gera automaticamente para o contentor — esse ficheiro é reescrito sempre que a rede do contentor é alterada pela interface do Proxmox, o que destruiria silenciosamente qualquer edição manual feita diretamente nele.

As rotas em falta foram também adicionadas no router OpenWRT (necessário para que a rede de casa soubesse devolver tráfego à rede da VPN), completando o encaminhamento nos três pontos: `vpnlab`, host Proxmox, e router.

Cliente 1: o tablet Android

O ficheiro `.ovpn` de cada dispositivo é montado por um pequeno script que junta as diretivas do cliente com os certificados/chaves relevantes, extraídos diretamente da PKI:

```
#!/bin/bash
set -e
EASYRSA_DIR=/etc/openvpn/easy-rsa
OUT=/root/craveiro-tablet.ovpn
{
echo "client"; echo "dev tun"; echo "proto udp"
echo "remote meulab.exemplo.duckdns.org 1194"
echo "resolv-retry infinite"; echo "nobind"
echo "persist-key"; echo "persist-tun"
echo "cipher AES-256-CBC"; echo "verb 3"; echo "key-direction 1"
echo "<ca>"; cat "$EASYRSA_DIR/pki/ca.crt"; echo "</ca>"
echo "<cert>"; awk '/BEGIN CERTIFICATE/,/END CERTIFICATE/' "$EASYRSA_DIR/pki/issued/craveiro-tablet.crt"; echo "</cert>"
echo "<key>"; cat "$EASYRSA_DIR/pki/private/craveiro-tablet.key"; echo "</key>"
echo "<tls-auth>"; cat /etc/openvpn/ta.key; echo "</tls-auth>"
} > "$OUT"
chmod 600 "$OUT"
```

Como transferir um ficheiro com chave privada sem o colar no chat

A lição do incidente de exposição anterior foi aplicada de forma sistemática a partir daqui: nunca colar conteúdo de um ficheiro com chave privada em lado nenhum. Em vez disso, um servidor HTTP temporário, ligado só o tempo necessário para uma transferência direta na rede local:

```
# No vpnlab
python3 -m http.server 8080 --bind 192.168.50.105
# No dispositivo de destino
wget http://192.168.50.105:8080/craveiro-tablet.ovpn
# De volta ao vpnlab, imediatamente after confirmado o import: Ctrl+C ao
servidor, e apagar o ficheiro de origem
rm -f /root/craveiro-tablet.ovpn /root/build-tablet-ovpn.sh
```

Dificuldade 1: "connection was interrupted" — NAT hairpin

A primeira tentativa de ligação, feita com o tablet *dentro* da própria rede de casa mas usando o domínio público (DuckDNS), falhou com "connection was interrupted". A causa é um problema clássico de NAT: muitos routers domésticos não conseguem redirecionar de volta, para dentro da própria rede, um pedido dirigido ao seu próprio endereço IP público — a isto chama-se falta de suporte para **NAT hairpin/loopback**.

Confirmado ao testar com o endereço IP interno do servidor VPN em vez do domínio público — ligou de imediato. A regra adotada a partir daqui: qualquer dispositivo que viva permanentemente dentro da rede de casa (como a futura VM Kali) usa desde logo o IP interno no seu perfil `.ovpn`, evitando este problema por completo; só dispositivos que se ligam também de fora de casa precisam do domínio público.

Dificuldade 2: ligado, mas "sem acesso à Internet"

Com a VPN ligada, o tablet reportava não ter acesso à Internet — nem sequer aos sites normais, e mesmo o ícone da própria rede WiFi (não da VPN) passou a mostrar "sem internet". Foram testadas, por ordem, três hipóteses:

1. **Seamless Tunnel** (uma opção específica do OpenVPN Connect no Android) — já estava desativada; não era a causa.
2. **DNS pushed inalcançável** — o servidor DNS entregue ao cliente (`8.8.8.8`) não estava dentro de nenhuma das rotas específicas que a VPN empurra para o cliente, ficando por isso inacessível enquanto ligado. Corrigido ao mudar para o AdGuard (`192.168.50.108`), que está dentro da rede tunelizada — esta era uma falha real, mas não explicava tudo.
3. **Comportamento interno da app Android** — a causa real. A app OpenVPN Connect para Android tem uma política interna anti-fuga de dados: sem uma diretiva completa de *redirect-gateway* (que redireciona *todo* o tráfego, não só o das rotas específicas), a app bloqueia deliberadamente todo o tráfego que não corresponda exatamente às rotas específicas recebidas do servidor — visível nos seus próprios registos como "block-ipv4"/"block-ipv6". Isto não são diretivas reais do protocolo OpenVPN (confirmado através de um pedido de funcionalidade em aberto no repositório oficial, mostrando que `--block-ipv4` nunca foi implementado como opção real do servidor) — são uma decisão de desenho do cliente Android, diferente do comportamento em clientes desktop/Linux, que fazem divisão de tráfego (*split-tunnel*) real ao nível do sistema operativo.

Aceite como comportamento normal, não como bug

Gestão, análise e todo o acesso ao laboratório funcionam perfeitamente através da VPN no tablet — só a navegação geral na Internet fica indisponível enquanto ligado. Como o propósito deste dispositivo na VPN é precisamente gerir o laboratório (não navegar), isto foi conscientemente aceite como funcionamento normal, sem necessidade de ativar um túnel completo (*redirect-gateway*) só para contornar esta limitação cosmética.

Cliente 2: a VM Kali (PC de ataque)

Foi criado um certificado individual dedicado, `craveiro-kali-vm`, com o perfil a apontar desde logo para o endereço IP interno do servidor (evitando pró-ativamente o problema de NAT hairpin já conhecido, já que esta VM vive sempre dentro de casa).

Dificuldade: a VPN caía sozinha, repetidamente

Pouco depois de ligar, a VPN desconectava-se sozinha, com o servidor a reportar "Inactivity timeout (--ping-restart), restarting" — apesar de o cliente continuar a enviar pacotes de manutenção da ligação (*keepalive*). O diagnóstico eliminou, um a um: o encaminhamento no Proxmox e no `vpnlab` (confirmados corretos com testes diretos de ligação entre eles); o firewall em ambos os lados (todas as regras em ACCEPT); e um aperto do intervalo de *keepalive* do servidor (de 10/120 para 5/30 segundos) ajudou a detetar o problema mais depressa, mas não o resolveu na origem.

A causa real: o adaptador de rede da VM estava em modo **NAT** no VMware — e o NAT virtual do VMware fecha sessões UDP inativas num intervalo de tempo muito curto, mais curto do que o próprio *keepalive* da VPN. O problema não estava na VPN nem na rede de casa — estava na camada de NAT do próprio hipervisor de desktop.

Mudança do adaptador de rede da VM de **NAT** para **Bridged**, nas definições do VMware Workstation Pro — a VM passou a ter um endereço IP próprio diretamente na rede física de casa, sem nenhuma camada de NAT intermediária. As desconexões cessaram por completo. Confirmado com um varrimento `nmap -Pn` à rede isolada através do túnel, sem interrupções:

```
nmap -Pn -p 22,80,3389 10.10.99.0/24
# 10.10.99.1 -> Proxmox (SSH aberto)
# 10.10.99.11 -> Metasploitable2 (SSH, HTTP abertos)
```

Frase-passe na chave privada: uma segunda camada de proteção

Um certificado válido garante que o dispositivo é reconhecido pelo servidor, mas, por defeito, qualquer pessoa que obtenha o ficheiro `.ovpn` (por exemplo, roubando o portátil ou o telemóvel) pode ligar-se imediatamente com ele. Para o certificado da VM Kali, foi pedido — e implementado — proteger a própria chave privada com uma frase-passe, exigida antes de qualquer ligação:

```
# Revogar o certificado sem frase-passe
easyrsa revoke craveiro-kali-vm
easyrsa gen-crl
systemctl restart openvpn@server

# Reemitir, desta vez SEM a opção "nopass" — pede e confirma uma frase-passe
easyrsa gen-req craveiro-kali-vm
easyrsa sign-req client craveiro-kali-vm
```

A partir daqui, ligar exige duas senhas em sequência: a do próprio sistema (`sudo`) e a frase-passe da chave da VPN:

```
cd ~
sudo openvpn --config craveiro-kali-vm.ovpn
# pede a password do sudo, depois "Enter PEM pass phrase:"
# aguardar por "Initialization Sequence Completed"
```

Isto resolve um problema de segurança real?

Sem frase-passe, qualquer pessoa com acesso ao ficheiro `.ovpn` liga-se de imediato — o risco real depende de quão fácil é alguém obter esse ficheiro (acesso físico ao dispositivo, cópia de segurança comprometida, etc.). Não é um risco crítico numa VM pessoal em casa, mas adicionar a frase-passe é uma correção de baixo custo para um cenário de perda ou acesso físico não autorizado — por isso foi aplicada aqui, e passa a ser a prática por defeito para novos certificados sensíveis.

O que falta: teste a partir de fora

Todo o encaminhamento e os dois níveis de encaminhamento de porta (Capítulo 4) foram confirmados como corretamente configurados, mas a VPN nunca foi ainda testada a ligar verdadeiramente **de fora** da rede de casa (via domínio público). Este teste está reservado para a fase de testes final (Capítulo 14), junto dos restantes cenários ponta-a-ponta.

Capítulo 10 — Alertas de Dispositivos e Resposta Automática

Duas camadas de resposta, dois níveis de rede

Detetar é só metade do trabalho de um SOC — a outra metade é decidir o que fazer com o que foi detetado. Este laboratório implementa duas respostas automáticas distintas, a níveis diferentes: uma no **router**, que vigia quem entra na rede (novos dispositivos), e outra no **Wazuh**, que vigia o que esses (ou outros) dispositivos fazem depois de estarem dentro (ataques detetados pelo Suricata). Ambas terminam da mesma forma: um alerta imediato por Telegram para o telefone do autor.

Deteção de dispositivos desconhecidos (no router)

Um script próprio, `check_new_devices.sh`, corre periodicamente no router OpenWRT (via `cron`, a cada 2 minutos) e compara a tabela ARP atual da rede **SENTINEL** (deliberadamente só esta — a SENTINELAB é, por definição, uma zona onde se esperam dispositivos desconhecidos, por isso não teria sentido alertar sobre eles ali) contra uma lista de endereços MAC conhecidos:

```
#!/bin/sh
# /etc/check_new_devices.sh (estrutura essencial)
KNOWN=/etc/known_macs.txt
ALERTED=/etc/alerted_macs.txt

for mac in $(ip neigh show | awk '{print $5}' | sort -u); do
    if ! grep -qi "$mac" "$KNOWN" && ! grep -qi "$mac" "$ALERTED"; then
        ip=$(ip neigh show | grep -i "$mac" | awk '{print $1}')
        curl -s -X POST "https://api.telegram.org/bot<TOKEN>/sendMessage" \
            -d chat_id=<CHAT_ID> \
            -d text="Novo dispositivo na SENTINEL: MAC $mac, IP $ip"
        echo "$mac" >> "$ALERTED"
    fi
done
```

```
# crontab do router
*/2 * * * * /etc/check_new_devices.sh
```

A lista `known_macs.txt` inclui todo o equipamento fixo do laboratório (host Proxmox, VM principal, Wazuh, honeypot, AdGuard) e os dispositivos pessoais habituais. A lista `alerted_macs.txt` evita repetir o mesmo alerta indefinidamente uma vez que um dispositivo novo já foi notificado.

Teste real, não simulado

Este sistema foi validado ligando o tablet Android à rede SENTINEL *antes* de o adicionar à lista de conhecidos — ou seja, um dispositivo genuinamente novo, nunca antes visto por esta rede. O alerta chegou corretamente ao Telegram, com o MAC e o IP reais do tablet. Só depois deste teste é que o seu MAC foi adicionado a `known_macs.txt`, para deixar de gerar alertas sobre ele.

Um MAC aleatório pode gerar falsos positivos

Muitos telemóveis modernos usam um endereço MAC aleatório e diferente por cada rede a que se ligam (uma funcionalidade de privacidade do próprio Android/iOS), o que poderia fazer este sistema alertar repetidamente sobre "novos" dispositivos que são, na realidade, o mesmo telemóvel de sempre. Ficou por confirmar formalmente se o telemóvel principal do autor mantém um MAC fixo especificamente na rede SENTINEL — um ponto a validar na fase de testes, para evitar fadiga de alertas.

Exposições acidentais do token do Telegram (e porque não há segredos neste capítulo)

Ao longo da construção deste sistema, o token do bot do Telegram foi exposto acidentalmente em texto simples **três vezes** — duas vezes por ter sido colado diretamente numa conversa de apoio, e uma terceira vez através do uso de `sh -x` (modo de depuração do shell, que imprime cada comando executado, incluindo variáveis com valores sensíveis) ao diagnosticar um problema. Em todas as três ocasiões, o token foi imediatamente revogado e substituído por um novo no BotFather (a ferramenta oficial de gestão de bots do Telegram), e os dois scripts que o usam (`check_new_devices.sh` no router, e `block_and_alert.sh`, abaixo) foram atualizados em sincronia. Por este motivo — e como norma geral para este manual — nenhum valor real de token, chave ou password aparece em lado nenhum deste documento, mesmo de forma parcial ou ofuscada.

Resposta automática a ataques (Active Response no Wazuh)

O Wazuh inclui um mecanismo chamado **Active Response**: a possibilidade de executar um script automaticamente sempre que um alerta cumpre determinados critérios, em vez de esperar por intervenção humana. Neste laboratório, está ligado especificamente aos alertas gerados pelo Suricata (Capítulo 6) com nível de severidade 6 ou superior:

```
<!-- ossec.conf, no manager -->
<active-response>
  <command>block_and_alert</command>
  <location>local</location>
  <rules_group>suricata</rules_group>
  <level>6</level>
</active-response>
```

```
#!/bin/bash
# /var/ossec/active-response/bin/block_and_alert.sh (estrutura essencial)
SRC_IP="$1"
iptables -I INPUT -s "$SRC_IP" -j DROP
iptables -I FORWARD -s "$SRC_IP" -j DROP
echo "$(date) BLOQUEADO: $SRC_IP" >> /var/log/sentinel-blocks.log
curl -s -X POST "https://api.telegram.org/bot<TOKEN>/sendMessage" \
  -d chat_id=<CHAT_ID> \
  -d text="IP bloqueado automaticamente: $SRC_IP (alerta Suricata)"
```

Ou seja, o percurso completo de uma deteção séria é: o Suricata identifica um padrão de ataque no tráfego → gera um alerta de nível ≥ 6 → o Wazuh recebe esse alerta e reconhece que corresponde à regra de Active Response → executa o script, que bloqueia imediatamente o IP de origem ao nível do firewall da própria VM Wazuh, regista o bloqueio, e notifica por Telegram — tudo sem intervenção humana no momento.

Ainda não testado com um ataque real

Este mecanismo foi implementado e o seu desenho verificado, mas o teste de ponta a ponta — lançar um ataque real contra um dos alvos vulneráveis a partir da VM Kali, e confirmar que o IP de ataque é de facto bloqueado automaticamente e que o alerta chega ao Telegram — está reservado para a fase de testes (Capítulo 14), já com a VM Kali e a VPN totalmente operacionais.

Capítulo 11 — SENTINEL AI: um Analista Assistente

O problema que este componente tenta resolver

Um SIEM como o Wazuh é excelente a agregar e mostrar alertas, mas interpretar esses alertas — perceber a história completa por trás de uma sequência de eventos, escrever um resumo claro, sugerir a próxima ação — continua a ser trabalho humano, e é normalmente o gargalo real num SOC, mesmo pequeno. O **SENTINEL AI** é uma tentativa de aliviar parte desse gargalo: um backend próprio que liga os alertas do laboratório a um modelo de IA, para produzir análise e resumos em linguagem natural.

O que é, tecnicamente

Corre num contentor LXC dedicado (CT101), como uma aplicação **FastAPI** — uma framework Python para construir APIs web modernas, escolhida pela sua simplicidade e pela geração automática de documentação interativa. É essa documentação automática, aliás, que serviu para localizar e confirmar o ponto de acesso ao serviço durante a preparação deste manual: o caminho raiz (/) devolve um erro 404 por não ter uma página própria definida, mas o caminho `/docs` — gerado automaticamente pelo FastAPI para qualquer API construída com esta framework — mostra uma interface Swagger completa e funcional, sem necessidade de qualquer configuração extra.

```
# Não funciona (sem rota definida para "/"):
http:// 192.168.50.101 :8000/

# Funciona sempre, automaticamente, em qualquer API FastAPI:
http:// 192.168.50.101 :8000/docs
```

Modos de análise

O backend expõe quatro modos distintos de análise assistida, cada um pensado para um tipo diferente de pergunta que um analista faria durante uma investigação: um modo de resumo rápido de um alerta isolado, um modo de correlação entre múltiplos eventos relacionados, um modo de explicação técnica (para quando o próprio analista quer entender melhor o mecanismo do ataque detetado), e um modo de geração de

relatório final, pensado para produzir o tipo de documento que fecha uma investigação — este último é o que se pretende usar no teste do fluxo completo Blue Team (Capítulo 14): do ataque, ao alerta do Suricata, à correlação no Wazuh, ao relatório final gerado pelo SENTINEL AI.

Capítulo 12 — Hack The Box: Prática Externa e Progressão Reconhecida

Porque não basta o laboratório próprio

O laboratório descrito até aqui é permanente, ilimitado no tempo, e totalmente controlado — vantagens claras para praticar sem pressa. Mas tem uma limitação inerente: qualquer progresso feito nele é invisível para o exterior. Numa área como a cibersegurança, onde se procura frequentemente demonstrar competência a terceiros (recrutadores, comunidade, colegas), ter também presença numa plataforma externa e reconhecida tem valor próprio — não substitui o laboratório pessoal, complementa-o.

O **Hack The Box** é uma plataforma online de treino ofensivo: aloja centenas de máquinas vulneráveis, geridas centralmente, acessíveis por VPN, com progressão estruturada, rankings públicos e badges — o tipo de credencial que um laboratório pessoal, por definição, não pode oferecer.

Ponto de entrada: Starting Point

Dentro da plataforma, a secção "Starting Point" é pensada especificamente para quem está a começar: três níveis (Tier 0 a Tier 2), cada um com máquinas guiadas passo a passo, de dificuldade crescente. A primeira máquina do Tier 0, "Meow", é deliberadamente trivial — tipicamente resolvida com um simples `nmap` para descobrir um serviço Telnet aberto, seguido de uma ligação direta com credenciais por defeito — servindo para validar todo o fluxo (ligar à VPN, encontrar o serviço, explorar, capturar a "flag") antes de progredir para desafios reais.

Duas formas de atacar: VPN própria vs. Pwnbox

A HTB oferece duas formas de interagir com as suas máquinas, sem relação nenhuma com a VPN deste laboratório (são infraestruturas completamente separadas, e nunca usadas em simultâneo neste projeto — uma serve para atacar as máquinas da HTB, a outra para gerir e analisar o laboratório pessoal):

- **VPN própria** — descarrega-se um ficheiro `.ovpn` específico da plataforma (por servidor/região), e liga-se através de uma máquina de ataque própria — neste caso, a mesma VM Kali usada no resto do laboratório (Capítulo 2), aproveitando as

ferramentas e configurações já lá instaladas: `sudo openvpn --config machines_eu-X.ovpn`.

- **Pwnbox** — uma máquina Kali completa, alojada pela própria HTB, acessível diretamente pelo browser, sem qualquer instalação ou VPN local. Mais simples de usar, mas o ambiente reinicia entre sessões (scripts e notas próprias não persistem).

A escolha para este laboratório foi usar a VPN própria com a VM Kali existente — mantém um único ambiente de trabalho, com ferramentas e notas persistentes entre sessões de ataque, tanto contra alvos do próprio laboratório como contra máquinas da HTB.

Porque manter VulnHub e Hack The Box, em vez de escolher só um

Não são alternativas concorrentes — servem propósitos diferentes e complementares. O VulnHub (Kioptrix, Metasploitable2 — Capítulo 8) oferece máquinas permanentes, sem pressa, ideais para prática repetida e para aprender com calma. A HTB oferece progressão reconhecida externamente, relevante sobretudo numa fase de procura de emprego em cibersegurança, onde um perfil com rank e badges visíveis é uma credencial que um laboratório pessoal não consegue substituir.

Ataques ainda não realizados

A conta, a VPN e o acesso à máquina "Meow" foram confirmados como funcionais, mas os ataques em si (a "Meow" e máquinas seguintes) ficam deliberadamente para a fase de testes/prática (Capítulo 14) — a prioridade, até este ponto, foi terminar toda a configuração do laboratório.

Capítulo 13 — Lista Consolidada de Erros e Correções

Este capítulo reúne, para consulta rápida, todos os erros e dificuldades descritos ao longo do manual — sem a narrativa completa de cada um, só o essencial: o que aconteceu, e como foi corrigido. Estão agrupados por área.

Router e rede

Login no router MEO falhava com meo/meo e admin/admin

Causa: este modelo (WiFi 6 / GEN8) não usa os defaults clássicos. **Correção:** credencial correta impressa em etiqueta física no equipamento.

Regra de encaminhamento de porta no menu errado

Causa: procurada inicialmente em "Segurança → Firewall e DMZ". **Correção:** a opção correta está em "Segurança → Acesso".

Interface de gestão do switch inacessível

Estado: ainda não resolvido — o endereço esperado do switch nunca respondeu. VLANs e espelhamento de porta nunca puderam ser configurados por esta via; o espelhamento de tráfego foi feito em software no Proxmox (Capítulo 6) como consequência direta.

Confusão recorrente: comandos "qm"/"pct" executados no sítio errado

Causa: estes comandos só existem no próprio host Proxmox — correr dentro de um contentor (ex: vpnlab) dá "command not found". **Correção:** confirmar sempre o prompt (deve mostrar o hostname do host Proxmox) antes de os executar; padrão que se repetiu várias vezes ao longo do projeto.

Proxmox e inventário

Kioptrix Level 1 em kernel panic ("No init found") ao arrancar

Causa: disco importado como SATA; o kernel antigo desta VM espera IDE. **Correção:** recriar o disco como `ide0` em vez de `sata0`.

DVWA "perdido" — contendor CT113 sem hostname próprio

Causa: hostname nunca personalizado após a criação inicial. **Correção:** localizado através de `pct list` + `pct config 113` + `pct exec 113 -- ls -la /var/www/html`, sem necessidade de adivinhar ou reinstalar.

VPN

Chave privada exposta em chat de apoio

Causa: conteúdo completo de um `.ovpn` colado por engano. **Correção:** certificado revogado e reemitido; descoberta em paralelo uma falha mais grave — a ausência da diretiva `crl-verify`, que tornava a revogação, por si só, inútil. Corrigida ao acrescentá-la ao `server.conf`.

"Connection was interrupted" ao ligar de dentro da própria rede

Causa: NAT hairpin/loopback — o router não redireciona de volta para dentro um pedido dirigido ao seu próprio IP público. **Correção:** usar o IP interno do servidor VPN para qualquer dispositivo que viva sempre dentro de casa.

Ligado à VPN, mas sem acesso à Internet geral

Causa (parcial): DNS entregue ao cliente (8.8.8.8) fora de qualquer rota tunelizada.
Causa real: comportamento interno do OpenVPN Connect para Android, que bloqueia todo o tráfego fora das rotas específicas na ausência de *redirect-gateway* completo.
Resolução: DNS corrigido para um servidor dentro da rede tunelizada; o bloqueio de internet geral foi aceite como comportamento normal, não como bug.

VPN da VM Kali a cair repetidamente ("--ping-restart")

Causa: adaptador de rede da VM em modo NAT no VMware — timeout de sessão UDP demasiado curto. **Correção:** mudar o adaptador para modo Bridged.

"Verify failure" ao gerar novo certificado com frase-passe

Causa: as duas introduções da frase-passe não coincidiram. **Correção:** repetir o comando com atenção — sem alteração de configuração necessária.

Encaminhamento IP desativado (ip_forward)

Causa: desativado tanto no contentor vpnlab como no host Proxmox — provavelmente nunca tinha funcionado. **Correção:** ativado e persistido nos dois (mecanismos diferentes em cada sítio, por particularidades do Proxmox).

Honeypots

Cowrie a correr sem o agente Wazuh instalado

Causa: honeypot instalado e a gerar eventos, mas nunca ligado ao SIEM. **Correção:** agente Wazuh instalado e registado, log JSON do Cowrie ligado, regras próprias criadas no manager.

Módulo SMB do OpenCanary sem alertas — bug em cadeia

Causas empilhadas: (1) permissões erradas no log de auditoria do Samba; (2) prefixo `full_audit` com menos campos do que o parser do OpenCanary espera; (3) esse desalinhamento causava uma exceção que matava silenciosamente o vigilante de ficheiro, sem erro visível. **Correção:** permissões corrigidas (`syslog:adm 640`) e prefixo ajustado para os 10 campos exatos esperados.

Processo e segurança operacional

Token do bot Telegram exposto três vezes

Causas: duas vezes colado em texto simples numa conversa de apoio, uma vez impresso por `sh -x` durante depuração. **Correção:** revogado e rodado no BotFather em cada ocasião, com atualização sincronizada de ambos os scripts que o usam.

Capítulo 14 — Próximos Passos: a Fase de Testes

Toda a configuração descrita nos capítulos anteriores está concluída e verificada peça a peça. O que falta, deliberadamente deixado para uma fase separada (Capítulo 1), é validar o sistema como um todo, em cenários reais de ponta a ponta. Esta secção descreve o que está planeado — não o que já aconteceu — e por isso não entra em detalhe sobre problemas, já que estes ainda não ocorreram.

Módulo Mobile: deteção de dispositivos comprometidos

Planeado como o último item antes de entrar verdadeiramente na fase de testes, para poder testar e aprender imediatamente a seguir. A ideia: ligar um smartphone à rede SENTINELAB (a zona isolada e sem filtragem, Capítulo 5), deixá-lo em repouso normal durante um período (por exemplo, 30 minutos), e depois analisar tudo o que tentou contactar — através dos registos do AdGuard (domínios) e de uma captura de tráfego com Wireshark (conteúdo e padrões de comunicação) — para procurar sinais de comportamento suspeito (comunicação com domínios desconhecidos, tráfego fora de horas, padrões de "beaconing" característicos de malware ou stalkerware). Está também considerada, como extensão opcional, a instalação do MobSF (Mobile Security Framework) para analisar diretamente aplicações (APKs) suspeitas encontradas no dispositivo.

Testes de ataque real

- **Fluxo Blue Team completo:** lançar um ataque real (nmap e exploração) contra o Metasploitable2 a partir da VM Kali, e confirmar que os alertas percorrem corretamente todo o caminho — Suricata deteta → Wazuh correlaciona → Active Response bloqueia o IP e notifica por Telegram → SENTINEL AI gera um relatório resumo.
- **Hack The Box:** resolver a máquina "Meow" e progredir pelo Tier 0 do Starting Point.
- **VulnHub:** continuar a série Kioptrix (1.1, 1.2, 2...) com dificuldade progressiva.

Validação externa

- **VPN a partir de fora de casa:** testar a ligação através do domínio público e dos dois encaminhamentos de porta (Capítulo 4), a partir de uma rede verdadeiramente externa (não apenas fora da própria casa via dados móveis, mas confirmando também o comportamento com o WiFi de outro local).
- **Repetir o teste do sistema de alerta de dispositivos desconhecidos** uma última vez, próximo da conclusão do projeto, como validação final antes de considerar esse mecanismo definitivamente fiável.

Itens de arquitetura ainda em aberto

- Diagnosticar e resolver o acesso à interface de gestão do switch (Capítulo 4), para eventualmente ativar VLANs e/ou espelhamento de porta ao nível do próprio switch.

Sobre este capítulo

Esta lista será substituída, numa versão futura deste manual, pelos resultados reais de cada teste — incluindo, tal como em todos os capítulos anteriores, qualquer dificuldade encontrada e a forma como foi corrigida.